Results

Results I

This section reports repository facts produced by the meta-project's live introspection pass. Test counts, module inventories, pipeline-stage counts, and figure data are injected from generated artifacts. Runtime and achieved-coverage claims are intentionally omitted when no versioned benchmark or coverage artifact backs them.

Multi-Project Pipeline Execution I

The `./run.sh` interactive orchestrator can execute the public roster through the shared DAG. The table below is a structural snapshot, not a timing benchmark: counts come from repository introspection, and the coverage column states each exemplar's declared policy floor rather than an unversioned observed percentage.

Multi-Project Pipeline Execution I

Project	Effective core stages[1]	Discovered tests	Declared project floor
`template_code_project`	8	263	90%
`template_prose_project`	8	165	90%
`template_autoresearch_project`	8	318	90%

Multi-Project Pipeline Execution I

[1]"Core-only" excludes LLM-tagged and other opt-in stages according to the YAML stage tags. A fresh run must be used to establish completion status or wall-clock performance for a particular machine and dependency set.

Infrastructure Test Suite I

Infrastructure Test Suite I

Metric	Value
Test files	922+
Total tests	~12,305
Infrastructure coverage gate	$\geq$60% configured floor
Prohibited mock-framework imports	Checked by the static no-mocks gate

Infrastructure Test Suite I

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

Infrastructure Module Inventory I

The introspection module (`template_template.core.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Infrastructure Module Inventory II

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	11			build_autoresearch_plan, readiness validation CLI
benchmark	4			Template harness scoring + comparative gates
config	0			Repository defaults + hardened templates
core	193			get_logger, load_config, TemplateError
docker	0			Containerisation scaffolding
doctor	14			Checkout diagnose/fix/undo repairs
documentation	23			FigureManager, build_api_index, generate_markdown_table
fonds	6			—
formal_contracts	5			—
llm	57			Ollama helpers, sanitization, review + translation pipelines
logrotate.d	0			Rotation snippets (documentation-first)
metadata	3			normalized_repository_url, generate_metadata_package
methods	10			build_methods_orchestration_plan, methods-stage contracts + validation
orchestration	14			PipelineRunner, entry point for ./run.sh

Infrastructure Module Inventory I

All 31 enumerated subdirectories carry Tier-1/`README.md` and Tier-2/`AGENTS.md` coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 `SKILL.md` descriptors for MCP routing (`infrastructure/skills` manifest generation).

Agentic Skill Documentation Coverage I

Agentic Skill Documentation Coverage I

Layer	Role
System prompts	Root `CLAUDE.md`, README policy
Structural	`AGENTS.md` directories
Skills	Optional `SKILL.md` manifests + generated `.cursor/skill_manifest.json`
PAI capsule	Repository level `PAI.md` narratives

Agentic Skill Documentation Coverage I

375+ Markdown shards under `docs/` capture operational knowledge without duplicating auto-generated inventories.

DAG Reference (Declarative Executor) I

Stages below mirror `pipeline.yaml` (executor-topological order—not strict numeric script filenames). Scripts live under `scripts/`.

DAG Reference (Declarative Executor) I

Name	Typical script / method	Responsibility	Failure semantics
Clean Output Directories	`_run_clean_outputs`	Deletes stale `output/` trees	Blocking
Environment Setup	`scripts/pipeline/stage_00_setup.py`	Validates tooling, PYTHONPATH scaffolding	Blocking
Infrastructure Tests	`scripts/pipeline/stage_01_test.py --infra-only`	Infra pytest + coverage gates	Tunable thresholds
Project Tests	`scripts/pipeline/stage_01_test.py --project-only`	Project pytest + coverage gates	Zero failures default
Project Analysis	`scripts/pipeline/stage_02_analysis.py`	Executes `projects/<name>/scripts/*.py`	Blocking
PDF Rendering	`scripts/pipeline/stage_03_render.py`	Pandoc → XeLaTeX manuscripts	Blocking
Output Validation	`scripts/pipeline/stage_04_validate.py`	Structural PDF/markdown probes	Blocking / warnings
LLM Scientific Review	`scripts/pipeline/stage_06_llm_review.py --reviews-only`	Local Ollama reviews	Skippable / exit 2 tolerated
LLM Translations	`scripts/pipeline/stage_06_llm_review.py --translations-only`	Optional translations	Skippable
Copy Outputs	`scripts/pipeline/stage_05_copy.py`	Mirrors deliverables → `output/<project>/`	Soft-fail surfaced in logs

DAG Reference (Declarative Executor) I

`scripts/pipeline/stage_07_executive_report.py` is **multi-project orchestration glue** invoked after iterating active projects—not a tenth DAG node for single-repo runs (`execute_pipeline.py`).

Steganographic Performance Boundary I

The steganography path exposes metadata injection, SHA-256 hashing, overlay generation, and optional QR-code insertion as separately testable operations. This revision makes no latency claim because the project does not currently generate a versioned benchmark artifact recording hardware, input PDF, warm-up policy, repetitions, and dispersion. A future performance table should be compiled from such an artifact rather than copied from an ad-hoc workstation run.

Rendered via `projects/templates/template_template` (`generate_manuscript_metrics.py` → injected tokens such as 31). Architecture figures stem from `template_template.figures.architecture_viz`—font sizes constrained by §QA.

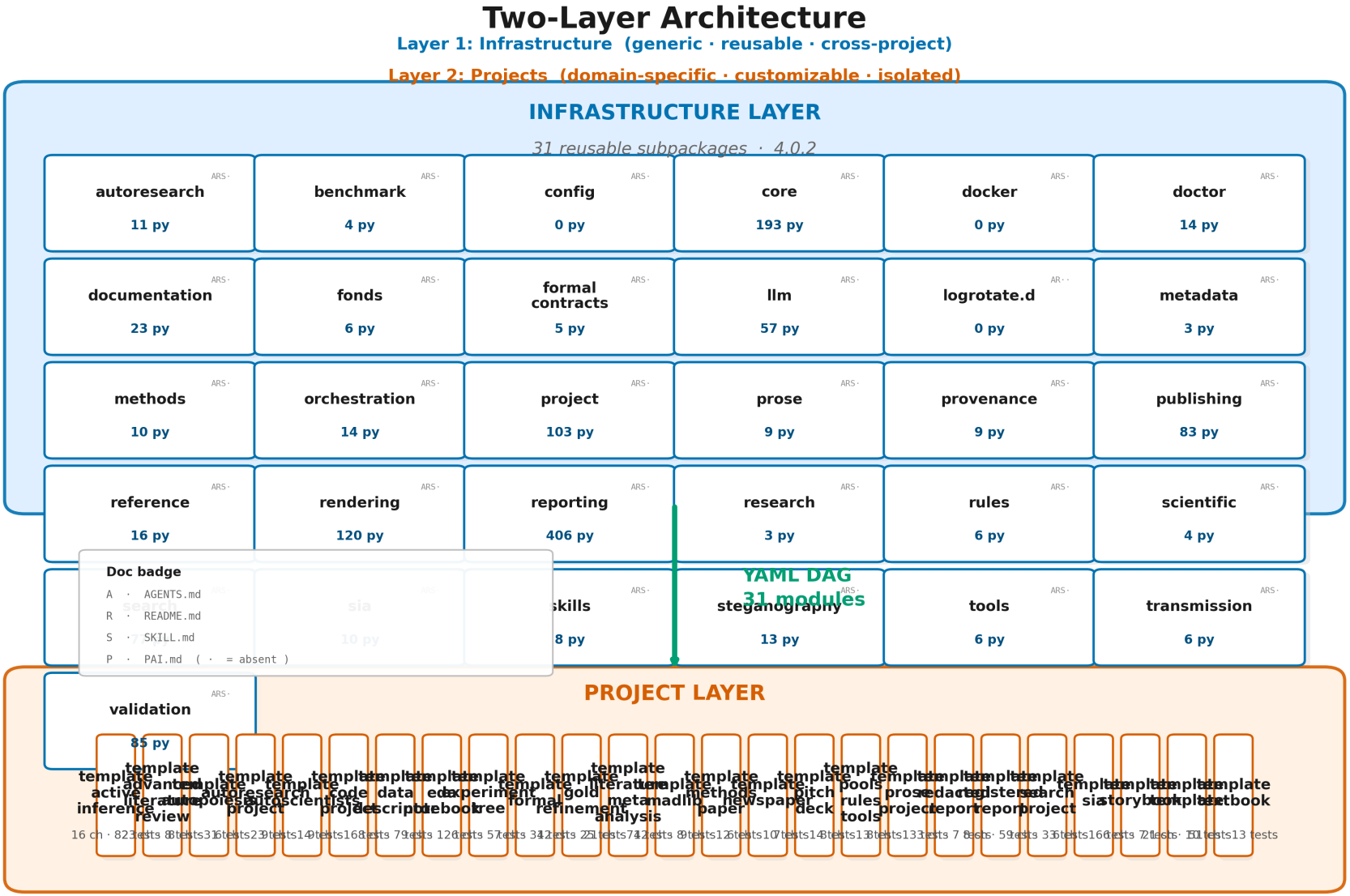


Figure 1. Live rendering of the Two-Layer Architecture from repository introspection: the infrastructure layer (top) holds the 31 reusable subpackages, each annotated with its Python file count and a four-slot documentation badge—A AGENTS.md, R

[ARSP]. A YAML DAG arrow connects it to the project layer (bottom) of public exemplars labelled with chapter and test counts. The takeaway: documentation-duality coverage is near-uniform across the infrastructure, and every box was placed from the same live data the prose cites.

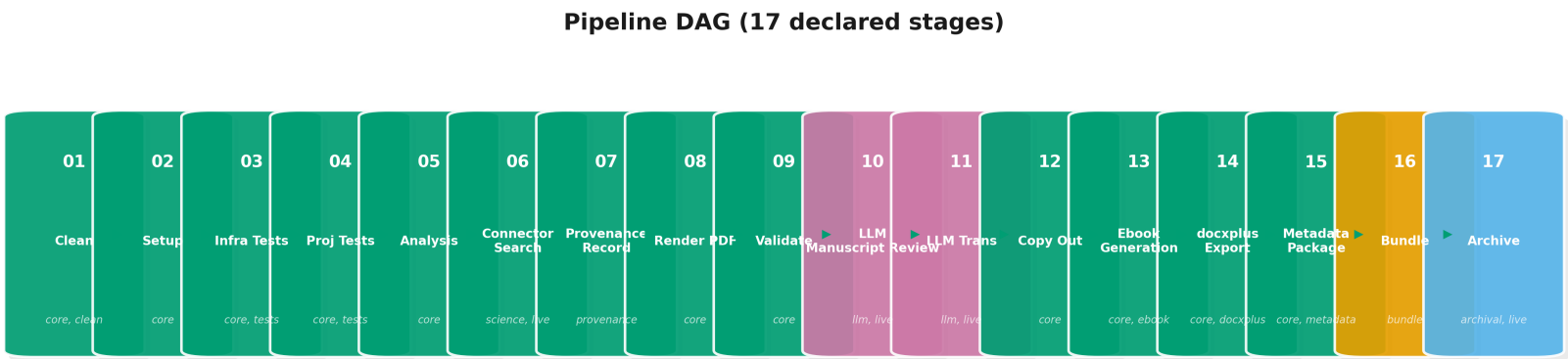
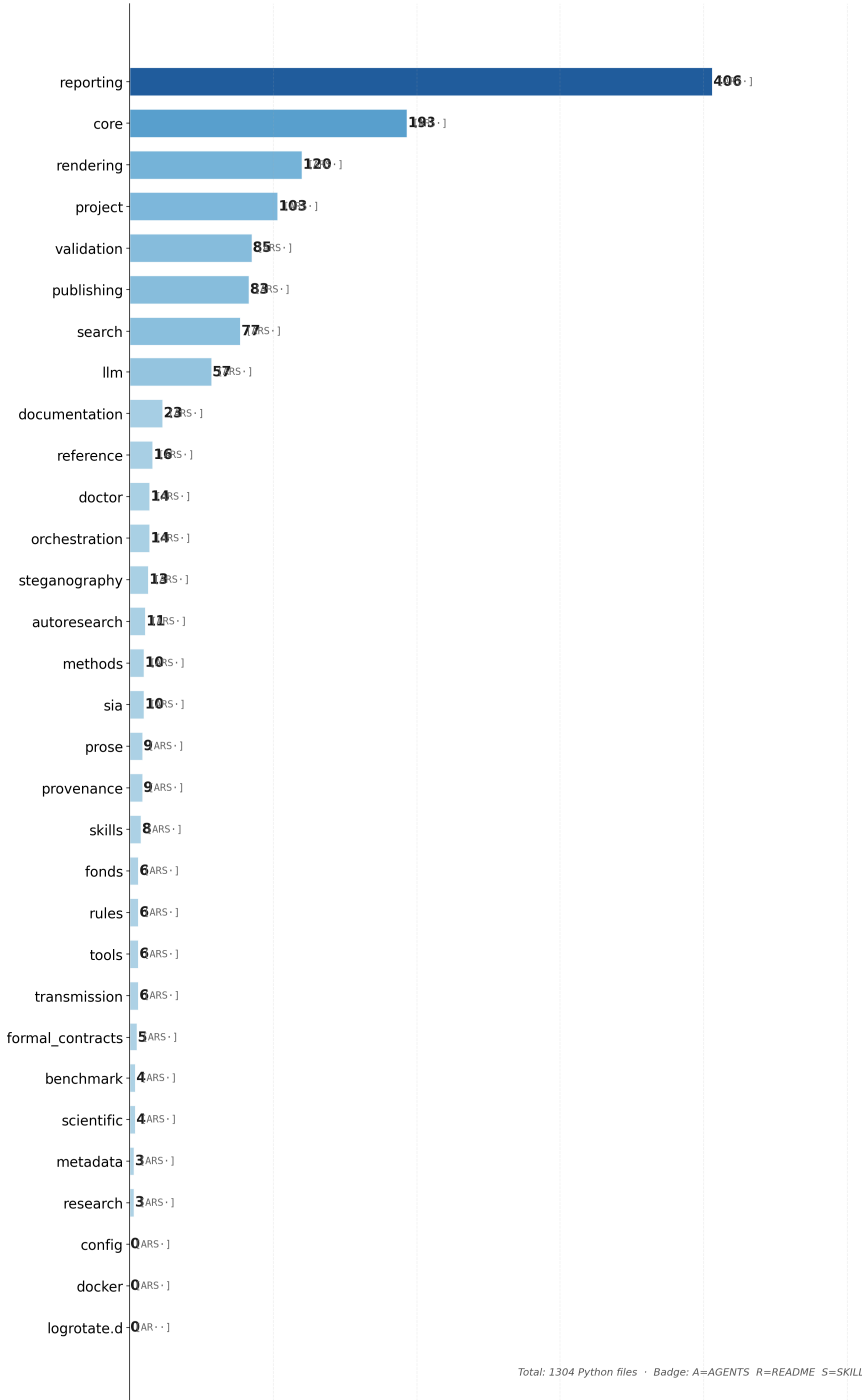


Figure 2. Pipeline DAG with 17 YAML-declared stages (core, LLM, ebook, docxplus, metadata, bundle, archival tags).

Self-Referential Analysis I

Infrastructure Module Inventory



Total: 1304 Python files · Badge: A=AGENTS R=README S=SKILL P=PAI

Comparative Feature Analysis I

Figure 4 summarizes the Appendix F capability matrix.



Figure 4. 14×10 heatmap annotated in appendix text—green full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

Comparative Feature Analysis I

[1] Nextflow lineage records exist at workflow scope, not per rendered PDF citation graph.

[2] DVC: content-addressed artifacts without native prose rendering.

[3] DVC: remote object stores (S3, GCS, Azure) without turnkey CI manuscript gates.

Test Quality Metrics I

Test Quality Metrics I

- **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- `template_code_project` focuses on numerical reproducibility assertions.
- `template_autoresearch_project` exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). `template_search_project` exercises literature-search workflows.